

SynWorld: Virtual Scenario Synthesis for Agentic Action Knowledge Refinement

Anonymous ACL submission

Abstract

In the interaction between agents and their environments, agents expand their capabilities by planning and executing actions. However, LLM-based agents face substantial challenges when deployed in novel environments or required to navigate unconventional action spaces. To empower agents to autonomously explore environments, optimize workflows, and enhance their understanding of actions, we propose SynWorld, a framework that allows agents to synthesize possible scenarios with multi-step action invocation within the action space and perform Monte Carlo Tree Search (MCTS) exploration to effectively refine their action knowledge in the current environment. Our experiments demonstrate that SynWorld is an effective and general approach to learning action knowledge in new environments.

1 Introduction

By leveraging decision-making capabilities to execute task-oriented actions within dynamic environments, Large Language Models (LLM) based agents demonstrate enhanced environmental interactivity and operational versatility (Song et al., 2023a; Liu et al., 2024b; Wu et al., 2025; Xi et al., 2025). In the real world, agents perform actions by leveraging tools like web search engines (Fan et al., 2024; Zhao et al., 2024a) or API calls (Liu et al., 2024a; Tao et al., 2024) to access feedback from the real world, which addresses the static knowledge limitations of LLMs, facilitating a deeper comprehension of the real world. It is crucial for agents to learn how to plan and execute actions in the environment. Nonetheless, as the complexity of tasks increases and novel environments emerge, manually annotated environment descriptions a predefined action documents (Qu et al., 2024; Sri et al., 2024) for agents are often not consistent with the actual environmental conditions and action usage (Liu et al., 2024d; Huang et al., 2024a). Refin-

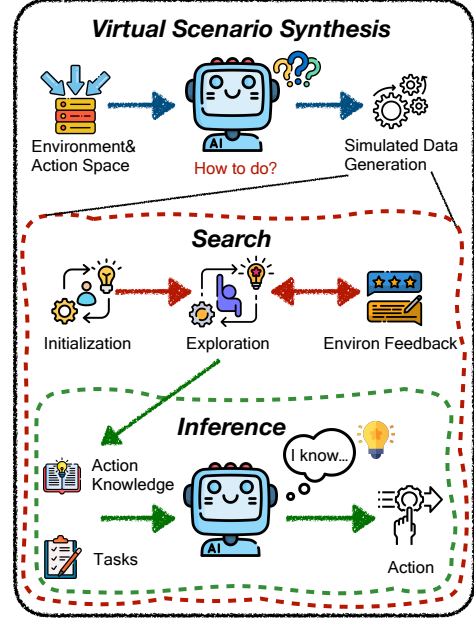


Figure 1: Our method with exploration to refine action knowledge in Synthesized Scenario.

ing accurate descriptions of the environment and actions is time-consuming and labor-intensive.

Therefore, to master inexperienced action and complicated task requirements in new complex environments, refinement of the agentic action knowledge is essential. Previous studies have explored the acquisition of action knowledge through feedback in single-action scenarios synthesized by llms. Similar to the way humans acquire skills through trial and error, agents can also optimize the descriptions of actions by leveraging feedback from simulated scenarios (Yuan et al., 2024; Du et al., 2024; Bouzenia et al., 2024). However, these methods exhibit two critical limitations: (1) Synthetic scenarios are often restricted to single-action scenarios, which hinders agents from learning workflows suitable for these tasks, and (2) The linear iterative optimization process lacks a clear direction for improvement, making it susceptible to stagnation and quickly reaching its performance ceiling.

To address these limitations, we propose a new framework, **SynWorld**, designed to assist agents in learning unfamiliar actions in new environments as shown in Figure 1. **SynWorld** firstly synthesizes more virtual scenarios involving multiple coordinated actions. Then through iterative MCTS optimization in the exploration of virtual scenarios, the framework achieves bidirectional refinement between action descriptions and workflow patterns to align with environmental constraints. Experiments demonstrate that action knowledge can be learned in virtual environments and generalized to the real world, effectively optimizing through MCTS exploration.

2 Background

2.1 Agent Planning

An agent interacts with its environment by perceiving its state, selecting actions to achieve a goal, and learning from feedback in the form of rewards. Its framework consists of a state space $\mathcal{S}$ that represents the environment’s properties, an action space $\mathcal{A}$ that defines allowable interactions, and an observation space Ω for perceptual inputs. Progress toward task T is measured through a reward function $\mathcal{R}$. Central to decision-making is a planning mechanism $\mathcal{P}_\theta$, where θ are fixed model weights. The agent’s architecture is defined by the tuple:

$$\mathcal{P}_\theta = \pi_\theta(\mathcal{S}, \mathcal{A}, \Omega, \mathcal{R}) \quad (1)$$

This formula delineates the manner in which an agent assesses its current state and interprets environmental feedback to generate plans.

2.2 Action Knowledge

Action knowledge $\mathcal{AK}$ serves as the strategic foundation governing an agent’s adaptive behavior in dynamic and unfamiliar environments. It contains **action description** about the awareness of executable actions with **cognitive workflows** about task decomposition and action sequences.

3 Method

In this section, we begin by detailing how to utilize the action space to synthesize scenarios and specific objectives. Subsequently, we dive into the application of MCTS to explore and discover action knowledge within these synthesized scenarios. The SynWorld framework is shown in Figure 2.

3.1 Scenario Synthesis

To address generalization challenges in multistep tool operationalization, we propose a framework that synthesizes scenarios through tool-conditioned task generation. Our methodology formalizes scenario synthesis as:

$$\mathcal{S}(t) = \{(\mathcal{B}, \mathcal{G}) \mid \forall t \subseteq T\}, \quad (2)$$

where a subset of tools t selected by llm from the complete set of tools T to design a scenario. Each scenario comprises two part: **Scenario $\mathcal{B}$** : The contextual scenario specifying initial conditions and constraints; **Goal $\mathcal{G}$** : The terminal objective requiring tool-mediated resolution. We provide examples using a few-shot approach to enable the llm to synthesize queries.

The mapping enforces that distinct tool combinations yield nontrivial scenario variations through systematic $\mathcal{B}$ - $\mathcal{G}$ pairings. Crucially, similarity constraints govern scenario relationships by preserving structural regularities across tool subsets – formally, if $t_i \sim t_j$ under a metric ϵ based on semantic similarity, their corresponding scenarios $(\mathcal{B}_i, \mathcal{G}_i)$ and $(\mathcal{B}_j, \mathcal{G}_j)$ maintain consistent parametric distances in the scenario space. This formulation enables the controlled generation of various scenarios.

$$t_i \sim t_j \implies d((\mathcal{B}_i, \mathcal{G}_i), (\mathcal{B}_j, \mathcal{G}_j)) < \epsilon. \quad (3)$$

3.2 Action Knowledge Exploration

Initialization The root node is initialized with predefined Action Knowledge, which serves as the foundation for task-solving logic. During the MCTS process, the UCB algorithm is used to select nodes, effectively balancing exploration and exploitation by choosing the node with the highest upper confidence limit.

Expansion Upon selecting node N_i as the candidate, an optimization process is initiated that retraces N_i to obtain insights from previous optimization experience $\mathcal{E}$. Each of these past optimization experiences $\mathcal{E}$ is composed of three elements: the pre-optimization score S_{before} , the post-optimization score S_{after} , and the modification $\mathcal{M}$ of the optimization actions taken.

$$\mathcal{E} = \{(S_{before}^i, S_{after}^i, \mathcal{M}^i) \mid N_i \in \text{Path}(N, N_0)\} \quad (4)$$

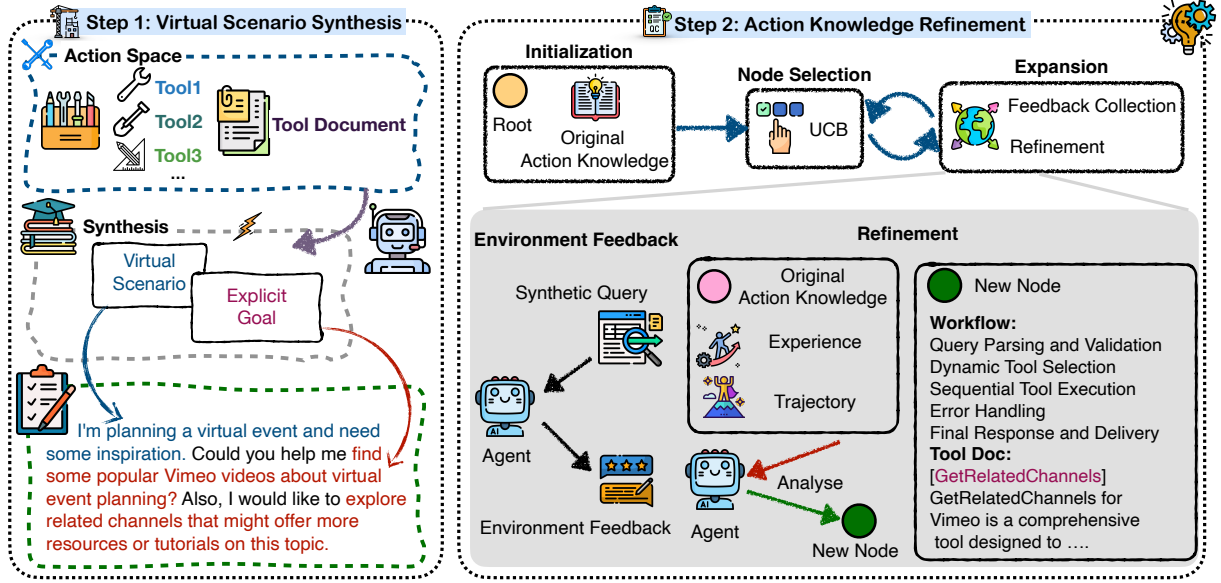


Figure 2: **The overall framework of SynWorld**: we first extract composable tools from the toolkit to generate new scenes and tasks. Then, we allow agents to explore the synthesized virtual scenes using MCTS to optimize action knowledge, thereby learning how to execute actions and plan tasks.

Based on the optimization experiences and exploration trajectories Tra from the past, the LLM-based agent π will analyze the discrepancies between the existing Action Knowledge and the environment. It will then optimize these to produce an updated version of the Action Knowledge.

$$\mathcal{AK}_{new} = \pi(\mathcal{AK}_{old}, \mathcal{E}, Tra) \quad (5)$$

Feedback Collection Once equipped with an optimized $\mathcal{AK}$, the agent π can explore the environment to perform tasks. For each individual task T , the agent interacts with the environment to receive feedback with the trajectory Tra_i and the final reward scores S_i . The score is related to the evaluation method of the task.

$$Tra_i, S_i = Env(\mathcal{AK}, \pi) \quad (6)$$

4 Experiment

4.1 Experiment Setup

Datasets and Baselines To demonstrate the efficiency of our approach in optimizing action knowledge, we selected two datasets: ToolBench (Qin et al., 2024) and HotpotQA (Yang et al., 2018), each offering unique challenges for a comprehensive evaluation. Following Qu et al. (2024), several strong methods are selected as our baselines, including ReAct (Yao et al., 2023), Self-Refine (Madaan et al., 2023), Easy-Tool (Yuan et al., 2024), and DRAFT (Qu et al., 2024). See detailed setting and evaluation in Appendix B.

Model	Method	ToolBench		HotpotQA
		PASS	WIN	
GPT-4-turbo	ReAct	50.67	67.00	54.61
	Self-Refine	56.80	73.00	55.85
	EasyTool	51.67	68.00	58.19
	DRAFT	54.83	72.00	57.71
	Ours	59.33	73.00	59.93
Qwen-long	ReAct	48.30	71.00	52.00
	Self-Refine	53.70	77.00	56.10
	EasyTool	50.80	63.00	58.34
	DRAFT	54.20	79.00	53.23
	Ours	57.20	81.00	59.91

Table 1: **Main results of SynWorld compared to other baselines** on ToolBench and HotpotQA. The best results of each model are marked in **bold**. PASS means the pass rate and WIN means the win rate of the trajectory compared to GPT-3.5-turbo in method of ReAct.

4.2 Main Results

For the task ToolBench that requires the combined use of multiple tools, as shown in Table 1, our approach achieved a PASS score of 59.33 and a WIN score of 73.00, marking a significant improvement compared to other methods for iterative optimization, demonstrating the advantages of our method in terms of tool combination and task planning optimization. **For the task HotpotQA that requires planning using a single tool and multi-step calls**, in the scenario where only a single tool is used but requires continuous multi-hop calls, our method has achieved state-of-the-art results. This indicates that we have not only aligned tool descriptions with the environment, but also succeeded in generating a generalizable planning workflow.

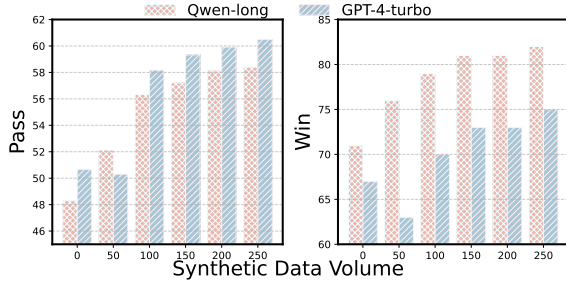


Figure 3: The variation in the pass rate of agents on the ToolBench in relation to the number of exploration scenarios.

4.3 Ablation Study

We observe that independently optimizing either the Workflow or the Tool Description using MCTS has its limitations in Table 2. We find that combining the optimization of both aspects leads to more effective results. An aligned Tool Description is beneficial for constructing a more reasonable Workflow, while a well-structured, general Workflow also enhances the exploration of tool usage. We believe that this synergy arises during the iterative optimization process, where the improved workflow can help identify tool usage that is closer to the correct trajectory, serving as strong negative examples to further refine the tool description. Conversely, a superior tool description enables the model to generate workflows that are more aligned with the environment.

Model	Method	Pass Rate
GPT-4-turbo	SynWorld	59.33
	w/o. Workflow	56.33 ^{-3.00}
	w/o. Description	53.16 ^{-6.17}
Qwen-long	SynWorld	57.20
	w/o. Workflow	57.00 ^{-0.20}
	w/o. Description	53.83 ^{-3.37}

Table 2: Ablation experiment results

4.4 Further Analysis

More simulated data enable precise virtual scenario synthesis, optimized action knowledge, and ultimately improved agent performance. In our experiments, we explore action knowledge by synthesizing a varying number of virtual scenarios. As shown in Figure 3, we find that as the number of scenarios synthesized increases, the performance of the Agent shows a corresponding upward trend. Specifically, within the range of 0 to 100 scenarios, the model’s performance continues

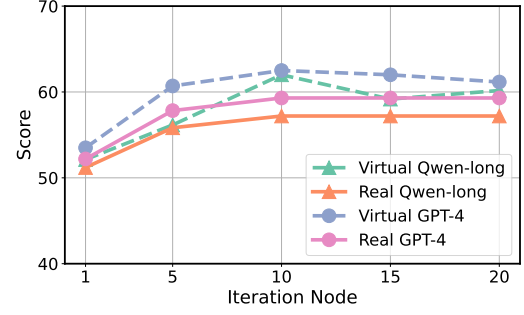


Figure 4: Changes in ToolBench pass rates in virtual and real-world scenarios with the number of iterative optimizations performed in the virtual environment.

to improve with the increase in the scenarios, indicating that action knowledge is indeed learnable. Although the rate of performance improvement slows down as the number of scenarios increases, the model’s performance remains on an upward trajectory. This phenomenon suggests that the process of learning action knowledge in the context of synthesized scenarios exhibits scalability.

Virtual scenario policies can be generalized to unseen environments and improved with iterations. By analyzing the relationship between action knowledge iterations and pass rates on Toolbench in both virtual and real environments, we find that the action knowledge gained in the virtual setting is generalizable and effective in real-world applications. Performance trends in both environments are similar in Figure 4. We observe a consistent upward trend in scores, particularly between 0 and 10 iterations, indicating that action knowledge can be optimized through environmental feedback. However, as iterations increase, the gains diminish, and we note slight declines in performance at times. This phenomenon is likely due to the limitations of exploring a fixed number of scenarios, where further iterations have less impact, and increasing complexity can hinder understanding.

5 Conclusion

In this paper, we propose SynWorld, a novel framework that synthesizes scenes that require multiple action steps and enhances agent action optimization through exploration in the synthetic virtual scenario. By systematically exploring diverse synthetic scenarios, our model achieves precise alignment between action descriptions and environmental contexts while identifying task-specific workflows suitable for tasks.

Limitations

We initially conduct empirical validation on two benchmarks: Toolbench (involving multi-tool calling scenarios) and HotpotQA (requiring multi-step action execution). While these demonstrate our method’s effectiveness, broader validation across diverse real-world applications remains valuable. Promising candidates include web-based search tasks, simulated environments and so on.

Our approach currently incurs non-trivial computational overhead due to the token-intensive virtual scenario synthesis process. The exploration phase further compounds this by exhaustively enumerating all possible scenarios. Future research should prioritize optimizing token efficiency through 1) developing more economical synthesis mechanisms for high-quality virtual scenarios and 2) establishing effective filtering criteria to identify the most pedagogically valuable scenarios.

The current action knowledge representation employs a purely text-based format. This presents opportunities to investigate alternative structured representations that could enhance reasoning capabilities, such as tabular organization of action parameters or executable code snippets encapsulating procedural knowledge.

References

Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. Repairagent: An autonomous, llm-based agent for program repair. *arXiv preprint arXiv:2403.17134*.

Yu Du, Fangyun Wei, and Hongyang Zhang. 2024. Any-tool: Self-reflective, hierarchical agents for large-scale api calls. *arXiv preprint arXiv:2402.04253*.

Zane Durante, Qiuyuan Huang, Naoki Wake, Ran Gong, Jae Sung Park, Bidipta Sarkar, Rohan Taori, Yusuke Noda, Demetri Terzopoulos, Yejin Choi, et al. 2024. Agent ai: Surveying the horizons of multimodal interaction. *arXiv preprint arXiv:2401.03568*.

Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A survey on RAG meeting llms: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2024, Barcelona, Spain, August 25-29, 2024*, pages 6491–6501. ACM.

Sihao Hu, Tiansheng Huang, Fatih Ilhan, Selim Tekin, Gaowen Liu, Ramana Kompella, and Ling Liu. 2024. A survey on large language model-based game agents. *arXiv preprint arXiv:2404.02039*.

Jerry Huang, Prasanna Parthasarathi, Mehdi Reza-gholizadeh, and Sarath Chandar. 2024a. Towards practical tool usage for continually learning llms. *arXiv preprint arXiv:2404.09339*.

Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. 2024b. Understanding the planning of llm agents: A survey. *arXiv preprint arXiv:2402.02716*.

Ziyan Jiang, Xueguang Ma, and Wenhui Chen. 2024. Longrag: Enhancing retrieval-augmented generation with long-context llms. *arXiv preprint arXiv:2406.15319*.

Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, et al. 2024. Personal llm agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459*.

Shilong Liu, Hao Cheng, Haotian Liu, Hao Zhang, Feng Li, Tianhe Ren, Xueyan Zou, Jianwei Yang, Hang Su, Jun Zhu, Lei Zhang, Jianfeng Gao, and Chunyuan Li. 2024a. Llava-plus: Learning to use tools for creating multimodal agents. In *Computer Vision - ECCV 2024 - 18th European Conference, Milan, Italy, September 29-October 4, 2024, Proceedings, Part XLVII*, volume 15105 of *Lecture Notes in Computer Science*, pages 126–142. Springer.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024b. Agent-bench: Evaluating llms as agents. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

Yanming Liu, Xinyue Peng, Yuwei Zhang, Jiannan Cao, Xuhong Zhang, Sheng Cheng, Xun Wang, Jianwei Yin, and Tianyu Du. 2024c. Tool-planner: Dynamic solution tree planning for large language model with tool clustering. *arXiv preprint arXiv:2406.03807*.

Zeyu Leo Liu, Shrey Pandit, Xi Ye, Eunsol Choi, and Greg Durrett. 2024d. Codeupdatearena: Benchmarking knowledge editing on API updates. *CoRR*, abs/2407.06249.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

366	Tula Masterman, Sandi Besen, Mason Sawtell, and Alex	<i>International Conference on Computer Vision, ICCV</i>	422
367	Chao. 2024. The landscape of emerging ai agent	2023, Paris, France, October 1-6, 2023, pages 2986–	423
368	architectures for reasoning, planning, and tool calling:	2997. IEEE.	424
369	A survey. <i>arXiv preprint arXiv:2404.11584</i> .		
370	Vaibhav Mavi, Anubhav Jangra, and Adam Jatowt. 2022.	Chan Hee Song, Jiaman Wu, Clayton Washington,	425
371	A survey on multi-hop question answering and gen-	Brian M Sadler, Wei-Lun Chao, and Yu Su. 2023b.	426
372	eration. <i>arXiv preprint arXiv:2204.09140</i> .	Llm-planner: Few-shot grounded planning for em-	427
373	Siqi Ouyang and Lei Li. 2023. Autoplan: Automatic	bodied agents with large language models. In <i>Pro-</i>	428
374	planning of interactive decision-making tasks with	<i>ceedings of the IEEE/CVF International Conference</i>	429
375	large language models . In <i>Findings of the Association</i>	<i>on Computer Vision</i> , pages 2998–3009.	430
376	<i>for Computational Linguistics: EMNLP 2023,</i>	S Deepika Sri, Raja CSP Raman, Gopinath Rajagopal,	431
377	<i>Singapore, December 6-10, 2023</i> , pages 3114–3128.	S Taranath Chan, et al. 2024. Automating rest	432
378	Association for Computational Linguistics.	api postman test cases using llm. <i>arXiv preprint</i>	433
379	Shuofei Qiao, Ningyu Zhang, Runnan Fang, Yujie Luo,	<i>arXiv:2404.10678</i> .	434
380	Wangchunshu Zhou, Yuchen Eleanor Jiang, Chengfei	Haotian Sun, Yuchen Zhuang, Ling kai Kong, Bo Dai,	435
381	Lv, and Huajun Chen. 2024. Autoact: Automatic	and Chao Zhang. 2023. Adapllanner: Adaptive plan-	436
382	agent learning from scratch via self-planning. <i>arXiv</i>	ning from feedback with language models. <i>Advances</i>	437
383	<i>preprint arXiv:2401.05268</i> .	<i>in neural information processing systems</i> , 36:58202–	438
384	Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan	58245.	439
385	Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang,	Chunliang Tao, Xiaojing Fan, and Yahe Yang. 2024.	440
386	Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian,	Harnessing llms for api interactions: A framework	441
387	Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li,	for classification and synthetic data generation. <i>arXiv</i>	442
388	Zhiyuan Liu, and Maosong Sun. 2024. Toolllm: Fa-	<i>preprint arXiv:2409.11703</i> .	443
389	cilitating large language models to master 16000+	Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao	444
390	real-world apis . In <i>The Twelfth International Con-</i>	Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang,	445
391	<i>ference on Learning Representations, ICLR 2024,</i>	Xu Chen, Yankai Lin, et al. 2024. A survey on large	446
392	<i>Vienna, Austria, May 7-11, 2024</i> . OpenReview.net.	language model based autonomous agents. <i>Frontiers</i>	447
393	Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai,	<i>of Computer Science</i> , 18(6):186345.	448
394	Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong	Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu,	449
395	Wen. 2024. From exploration to mastery: En-	Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim.	450
396	abling llms to master tools via self-driven interac-	2023a. Plan-and-solve prompting: Improving zero-	451
397	tions . <i>CoRR</i> , abs/2410.08197.	shot chain-of-thought reasoning by large language	452
398	Shreyas Sundara Raman, Vanya Cohen, Eric Rosen,	models. <i>arXiv preprint arXiv:2305.04091</i> .	453
399	Ifrah Idrees, David Paulius, and Stefanie Tellex. 2022.	Ruoyao Wang, Peter A. Jansen, Marc-Alexandre Côté,	454
400	Planning with large language models via corrective	and Prithviraj Ammanabrolu. 2022. Scienceworld: Is	455
401	re-prompting. In <i>NeurIPS 2022 Foundation Models</i>	your agent smarter than a 5th grader? In <i>Proceedings</i>	456
402	<i>for Decision Making Workshop</i> .	<i>of the 2022 Conference on Empirical Methods in</i>	457
403	Weizhou Shen, Chenliang Li, Hongzhan Chen, Ming	<i>Natural Language Processing, EMNLP 2022, Abu</i>	458
404	Yan, Xiaojun Quan, Hehong Chen, Ji Zhang, and Fei	<i>Dhabi, United Arab Emirates, December 7-11, 2022,</i>	459
405	Huang. 2024. Small llms are weak tool learners: A	pages 11279–11298. Association for Computational	460
406	multi-llm agent. <i>arXiv preprint arXiv:2401.07324</i> .	Linguistics.	461
407	Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté,	Zihao Wang, Shaofei Cai, Anji Liu, Xiaojian Ma, and	462
408	Yonatan Bisk, Adam Trischler, and Matthew J.	Yitao Liang. 2023b. Describe, explain, plan and	463
409	Hausknecht. 2021. Alfworld: Aligning text and em-	select: Interactive planning with large language mod-	464
410	bodied environments for interactive learning . In <i>9th</i>	els enables open-world multi-task agents . <i>CoRR</i> ,	465
411	<i>International Conference on Learning Representa-</i>	abs/2302.01560.	466
412	<i>tions, ICLR 2021, Virtual Event, Austria, May 3-7,</i>	Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten	467
413	<i>2021</i> . OpenReview.net.	Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou,	468
414	Simranjit Singh, Andreas Karatzas, Michael Fore, Irak-	et al. 2022. Chain-of-thought prompting elicits rea-	469
415	lis Anagnostopoulos, and Dimitrios Stamoulis. 2024.	soning in large language models. <i>Advances in neural</i>	470
416	An llm-tool compiler for fused parallel function call-	<i>information processing systems</i> , 35:24824–24837.	471
417	ing. <i>arXiv preprint arXiv:2405.17438</i> .	Jialong Wu, Wenbiao Yin, Yong Jiang, Zhenglin Wang,	472
418	Chan Hee Song, Brian M. Sadler, Jiaman Wu, Wei-Lun	Zekun Xi, Runnan Fang, Deyu Zhou, Pengjun	473
419	Chao, Clayton Washington, and Yu Su. 2023a. Llm-	Xie, and Fei Huang. 2025. Webwalker: Bench-	474
420	planner: Few-shot grounded planning for embodied	marking llms in web traversal. <i>arXiv preprint</i>	475
421	agents with large language models . In <i>IEEE/CVF</i>	<i>arXiv:2501.07572</i> .	476

- Zekun Xi, Wenbiao Yin, Jizhan Fang, Jialong Wu, Runnan Fang, Ningyu Zhang, Jiang Yong, Pengjun Xie, Fei Huang, and Huajun Chen. 2025. [Omnithink: Expanding knowledge boundaries in machine writing through thinking](#). *Preprint*, arXiv:2501.09751.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. [Hotpotqa: A dataset for diverse, explainable multi-hop question answering](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, pages 2369–2380. Association for Computational Linguistics.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. [React: Synergizing reasoning and acting in language models](#). In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.
- Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Yongliang Shen, Kan Ren, Dongsheng Li, and Deqing Yang. 2024. [EASYTOOL: enhancing llm-based agents with concise tool instruction](#). *CoRR*, abs/2401.06201.
- Siyun Zhao, Yuqing Yang, Zilong Wang, Zhiyuan He, Luna K Qiu, and Lili Qiu. 2024a. Retrieval augmented generation (rag) and beyond: A comprehensive survey on how to make your llms use external data more wisely. *arXiv preprint arXiv:2409.14924*.
- Yuyue Zhao, Jiancan Wu, Xiang Wang, Wei Tang, Dingxian Wang, and Maarten De Rijke. 2024b. Let me do it for you: Towards llm empowered recommendation via tool learning. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1796–1806.
- Wangchunshu Zhou, Yuchen Eleanor Jiang, Long Li, Jialong Wu, Tiannan Wang, Shi Qiu, Jintian Zhang, Jing Chen, Ruipu Wu, Shuai Wang, et al. 2023. Agents: An open-source framework for autonomous language agents. *arXiv preprint arXiv:2309.07870*.
- Wangchunshu Zhou, Yixin Ou, Shengwei Ding, Long Li, Jialong Wu, Tiannan Wang, Jiamin Chen, Shuai Wang, Xiaohua Xu, Ningyu Zhang, et al. 2024. Symbolic learning enables self-evolving agents. *arXiv preprint arXiv:2406.18532*.
- Yuqi Zhu, Shuofei Qiao, Yixin Ou, Shumin Deng, Ningyu Zhang, Shiwei Lyu, Yue Shen, Lei Liang, Jinjie Gu, and Huajun Chen. 2024. Knowagent: Knowledge-augmented planning for llm-based agents. *arXiv preprint arXiv:2403.03101*.

A Related Works

A.1 Agent Planning

Recent studies has shown that in the realm of complex task-solving (Ouyang and Li, 2023; Sun et al., 2023; Liu et al., 2024c), the capacity for planning and refinement within large models has become increasingly pivotal. It has marked a transition from early Methods like CoT(Wei et al., 2022), Plan and Solve (Wang et al., 2023a), which tackle tasks sequentially, to the sophisticated agentic workflows of today, where model planning is instrumental in addressing a myriad of complex tasks, including question answering (QA) (Mavi et al., 2022), embodied interaction (Yao et al., 2022), tool invocation (Masterman et al., 2024), and long-form text generation (Jiang et al., 2024).

However, initial planning efforts are fraught with deficiencies due to the complexity of environments. When faced with unfamiliar environments, relying solely on human-written task descriptions without interaction with the environment often leads to plans that are misaligned with the actual tasks, or plans that seem reasonable but fail during execution due to a lack of accurate action knowledge. Consequently, there has been a surge in research (Song et al., 2023b; Wang et al., 2023b) focused on refining plans and workflows. These efforts typically leverage direct environmental feedback or design a reward score for end-to-end plan correction, but they often lack a medium for the intermediate processes, which obscures the transparency of the plan refinement process. Moreover, the refinement of plans and the collection of feedback are usually linear and iterative (Qu et al., 2024), resulting in low efficiency and a lack of diversity.

A.2 Knowledge-Augmented Agents

LLMs, as agents interacting with specific environments, often need to provide action signals to these environments (Zhou et al., 2023, 2024; Durante et al., 2024). These action signals can be either restricted or open actions related to the environment (Wang et al., 2024; Li et al., 2024; Hu et al., 2024). For instance, they might involve specific movements in embodied task scenarios or the use of various tools like search or OCR in tool invocations. By incorporating these actions, on one hand, the Agent gains the ability to interact with the environment, allowing LLMs to transcend mere textual output (Shridhar et al., 2021; Wang et al., 2022). On the other hand, these external actions endow

the Agent with a capability similar to humans using tools, compensating for the inherent limitations of LLMs, such as search tools that can alleviate issues of knowledge hallucination or obsolescence in LLMs (Singh et al., 2024; Zhao et al., 2024b; Shen et al., 2024).

Current methods for learning action knowledge are mainly divided into two categories: one involves creating a large amount of synthetic data to construct trajectories for executing actions to train the model (Qiao et al., 2024; Huang et al., 2024b; Zhu et al., 2024), which is costly and has poor generalizability across different tasks; the other relies on prompt engineering (Raman et al., 2022), placing explicit action knowledge about how to plan to execute actions within the prompt, and then using ICL (In-Context Learning) methods to enable the model to learn to invoke these actions. While convenient, these methods can be inaccurate as the artificially constructed planning knowledge may not accurately reflect the true state of the environment, leading to potential biases.

B Setting

B.1 Datasets

ToolBench contains tasks using over 16,000 RapidAPI tools. It assesses a model’s ability to plan and execute complex workflows.

HotpotQA is a multi-hop QA dataset with questions requiring multiple steps to answer. We employ Google Search as the search engine in the experiment.

B.2 Evaluation

ToolBench: Evaluated using pass rate and win rate. We record planning steps and tool invocations, then submit the trajectory for assessment. Win rate is compared to React’s performance. **HotpotQA:** Evaluated using F1 score, comparing model answers to gold answers (reward 0–1). These datasets and metrics allow us to rigorously validate our approach across varied contexts.

B.3 Baselines

Several strong methods are selected as our baselines, including: ReAct (Yao et al., 2023) which interacts with environment to reason the next step, Self-Refine (Madaan et al., 2023) which uses the feedback from environment to refine the origin prompt, Easy-Tool (Yuan et al., 2024) which uses llm firstly to refine the tool description and

then break down the tasks to complete them, and DRAFT (Qu et al., 2024) to synthesize tasks on a single tool for exploration to learn how to use the tool.

B.4 Experiment Setup

The backend model used in our experiments is Qwen-Long-0916, while the version of GPT-4 is 0613. The token usage in our method is approximately 6-8 million tokens. We configured the width of MCTS to 3 and set the similarity threshold to 0.6. After balancing effectiveness and cost, we synthesized 200 scenes and conducted 15 iterations on them during the experiment.

C Prompt Template

See in Table 3, 4

Prompt for Tool Description in Action knowledge

Analyze the following tool execution trajectories to improve tool interface documentation.

For all trajectories:

1. Identify functional mismatches between original description and actual usage patterns
2. Detect parameter inefficiencies (missing/underutilized fields)
3. Extract implicit requirements from error patterns
4. Generate enhanced documentation with:
Clear input specifications (required vs optional)
Contextual usage guidelines
Error prevention tips
Response format expectations

Here is an **example**.

Now it's your turn to analyze the following tool execution trajectories to improve tool interface documentation.

tool_name: **tool_name**

original_description: **original_description**

trajectory: **trajectory**

Please provide your Optimize Description for the tool. Just modify the description part and do not change the parameters description.

Make Sure your description is clear and concise.

Table 3: Prompt used for tool document refinement.

Prompt for Workflow in Action knowledge

Analyze the provided interaction trajectory and existing workflow steps to derive a generalized, reusable workflow for similar tool calling tasks.

1. Analyzing error patterns (authentication gaps, deprecated endpoints) and tool dependencies from interaction histories.
2. Extracting implicit requirements (authentication, sorting logic) and mandatory parameters from error responses.
3. Structuring a generic workflow with authentication validation, parameter checks, state management between API calls, and error fallbacks.

Here is an **example**.

Now it's your turn.

Existing Workflow: **workflow**

Trajectory: **trajectory**

Please provide your Optimize Workflow for the task. And make sure your workflow is clear and concise and no longer than 200 words.

Table 4: Prompt used for workflow generation.